

Day3 · 19차시

프롬프트 관리 (PromptOps)

프롬프트는 코드에 박는 문자열이 아니라, 버전으로 관리하는 자산.
Langfuse Prompt Management로 등록 → 배포(라벨) → 롤백까지.

SECTION 00

오프닝 — 한 줄 고쳤다가 무너졌다

“프롬프트 한 줄 고쳤다가 품질이 무너졌다 — 어떻게 안전하게 바꾸고 즉시 되돌릴까?”

— 하드코딩된 프롬프트는 추적·롤백·비교가 안 된다. 그래서 관리가 필요하다.

관찰했으면, 이제 안전하게 바꾼다

▶ 관찰하다 보면 결국 '프롬프트를 고쳐야겠다'에 도달한다 — 그런데 그 변경도 배포만큼 위험하다

18차시 — 관찰

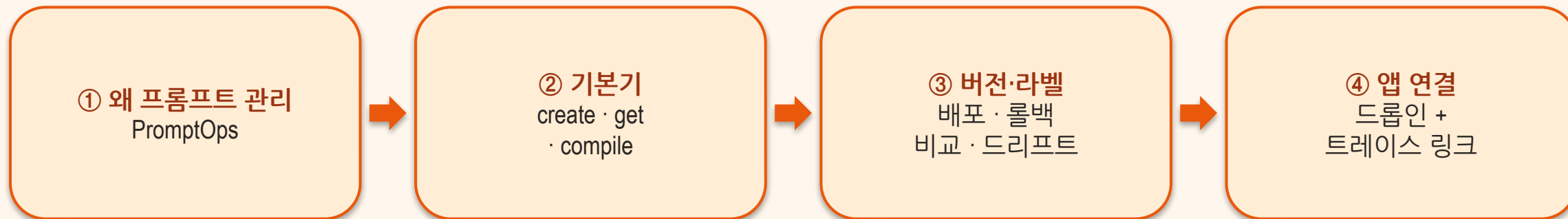
- trace·score로 운영 상태를 봤다
- score 하락을 감지했다
- 질문: “무엇이 문제인가?”

VS

19차시 — 안전한 변경

- 운영의 핵심 입력 = **프롬프트**
- 버전·라벨로 배포하고 롤백한다
- 질문: “어떻게 안 무너지게 바꾸나?”

이 차시 동안 얻는 것



SECTION 01

왜 프롬프트 관리인가 (PromptOps)

하드코딩된 프롬프트의 문제

▶ 코드 곳곳에 흩어진 문자열 → 추적·롤백·비교 불가

❌ 코드에 박은 문자열

- 누가·언제·왜 바꿨는지 모름
- 롤백 불가
- A/B·비교 불가
- 배포 = 코드 재배포

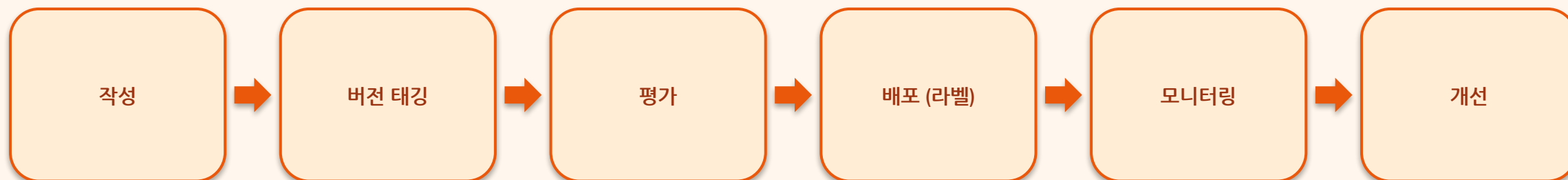
VS

✅ 이름 붙은 버전 자산

- 변경 이력이 남는다
- 라벨만 옮겨 즉시 롤백
- 같은 셋으로 A/B 비교
- 앱 재배포 불필요

프롬프트를 코드처럼 관리하는 운영

▶ '한 줄 바꿨더니 무너졌다'를 막는 안전장치 — 평가 단계는 20차시에서



🔄 PromptOps 개선 루프

프롬프트 관리의 4요소

▶ 이 4개가 Langfuse 프롬프트 관리의 데이터 모델 그대로

이름 (name)

recommend-system

버전 (version)

v1 · v2 · v3 ...

라벨 (label)

production · latest

변수 (variable)

{{area}} · {{budget}}

프롬프트 카탈로그에 남기는 것들

▶ 메타데이터가 풍부해야 발견·감사(거버넌스)가 된다

이름·버전

spam-filter-v2처럼 식별 가능하게

모델 호환성

어떤 모델·파라미터와 세트인가 (config)

승인 상태

실험 중 / 검토 대기 / production 승인

성능 기준선

이 버전의 평가 점수 — 다음 버전과 비교할 잣대

작성자·배포 이력

누가 만들고 언제 배포됐나

SECTION 02

Langfuse 프롬프트 관리 기본

Langfuse 프롬프트 관리란

▶ UI/SDK로 등록·편집·버전 비교·배포 — 트레이싱·평가와 같은 플랫폼

버전들

v1 · v2 · v3 — 저장할 때마다 자동으로 쌓인다

라벨

production · latest — '지금 쓰는 버전'을 가리키는 포인터

config

모델·temperature를 프롬프트와 **한 버전으로** 저장

사용 트레이스

이 프롬프트를 쓴 호출 기록 → 관찰성(18차시)과 연결

실제 화면

프롬프트 카탈로그 → 버전·라벨·변수

▶ 왼쪽: 등록된 프롬프트 목록 · 오른쪽: recommend-system의 v1→v2(production) + 변수 {{context}}·{{query}}

Name	Versions	Type	Latest Version Created At	Number of Observations (7d)	Tags	Actions
recommend-system	3	chat	2026-07-10 12:40:46	24		

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse 31k

Settings

Book a call

Support

워크샵 강사 workshop@exam...

Rows per page 50 Page 1 of 1

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse 31k

Settings

Book a call

Support

워크샵 강사 workshop@exam...

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse 31k

Settings

Book a call

Support

워크샵 강사 workshop@exam...

create_prompt (text / chat)

▶ config에 모델·파라미터를 프롬프트와 함께 저장 — '그때 그 결과'의 재현성

```
from langfuse import get_client
langfuse = get_client()
langfuse.create_prompt(
    name="recommend-system", type="text",
    prompt="너는 맛집 추천 도우미다. {{area}} 예산 {{budget}}원 이하로 추천하라.",
    labels=["production"],
    config={"model": "gpt-4o-mini", "temperature": 0.7})
```

get_prompt · compile — 앱은 이름만 안다

▶ 프롬프트 본문이 바뀌어도 앱 코드는 한 줄도 안 바뀐다 — 결합이 끊어지는 순간

```
prompt = langfuse.get_prompt("recommend-system")      # 기본 label="production"  
system = prompt.compile(area="강남", budget=20000)    # {{변수}} 치환  
# prompt.config["model"] 로 모델도 함께 사용
```

text vs chat + config

▶ few-shot 예시까지 한 세트로 관리하려면 chat

text 프롬프트

- 단일 문자열
- system/user에 삽입
- 간단한 지시에 적합

VS

chat 프롬프트

- role 배열 전체
- messages 구조를 버전 관리
- 멀티턴·역할 분리·few-shot 세트

few-shot 예시 관리

▶ 예시는 프롬프트의 절반 — 본문과 함께 버전 관리한다

무엇을

프롬프트 안에 심는 입력→출력 예시(in-context examples)

왜 관리하나

어떤 예시가 품질을 끌어올렸는지 추적·교체·회수가 필요

어떻게

chat 프롬프트의 messages로 함께 버전 관리 — 예시 교체 = 새 버전

SECTION 03

버저닝·라벨·배포·롤백

저장마다 새 버전 · 라벨이 '지금'을 가리킨다

▶ git으로 치면 — 버전은 커밋, 라벨은 움직이는 태그

v3

latest — 방금 저장한 최신

v2

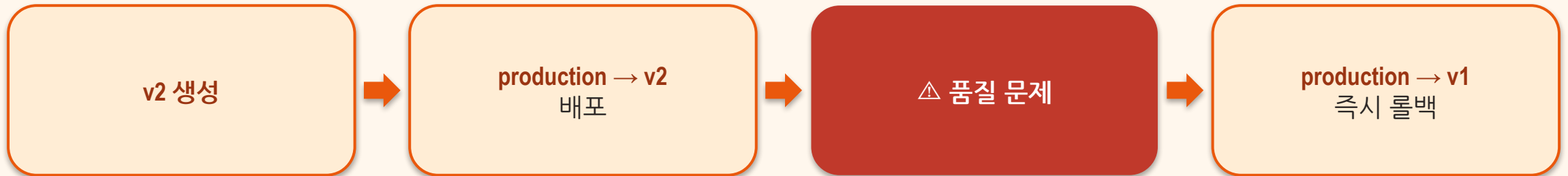
← production (지금 쓰는 버전)

v1

이전 버전 — 언제든지 돌아갈 수 있다

배포 = 라벨 이동 / 롤백 = 라벨 되돌리기

▶ 앱 재배포 불필요 — 오픈닝 질문의 답이 이 라벨 이동



Champion vs Challenger — 느낌이 아니라 통계로

▶ 책의 실험: 길고 정교한 프롬프트(B)가 오히려 졌다 — 경험주의가 직관을 이긴다

Champion — v1 (현재)

- 정밀도 평균 0.837 (± 0.037)
- 같은 평가셋으로 반복 측정
- 기준선(baseline)

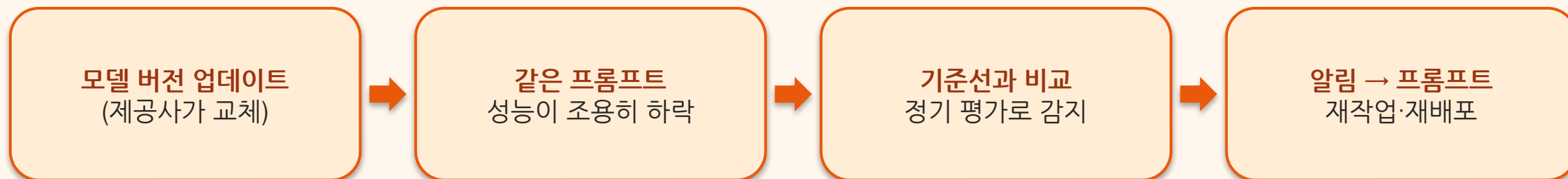
VS

Challenger — v2 (후보)

- 정밀도 평균 0.776 (± 0.031)
- 더 길고 정교했지만 낮았다
- t-검정 $p < 0.001 \rightarrow$ 기각

모델이 바뀌면 프롬프트도 흔들린다 (Prompt Drift)

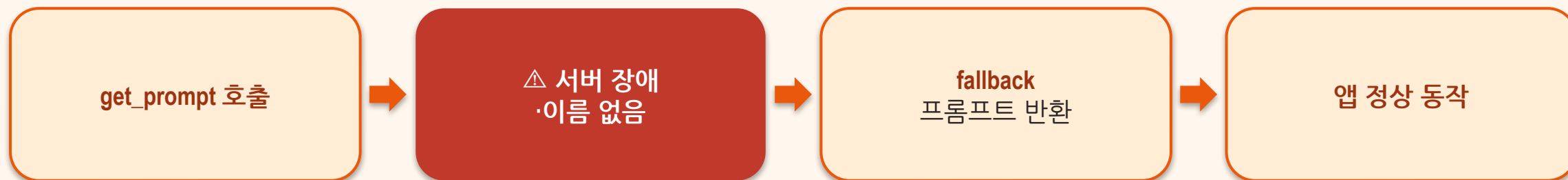
▶ 내가 안 바뀌도 무너질 수 있다 — 기준선과 정기 평가가 안전망



🔗 프롬프트 드리프트 — 16차시 '조용한 실패'의 프롬프트 버전

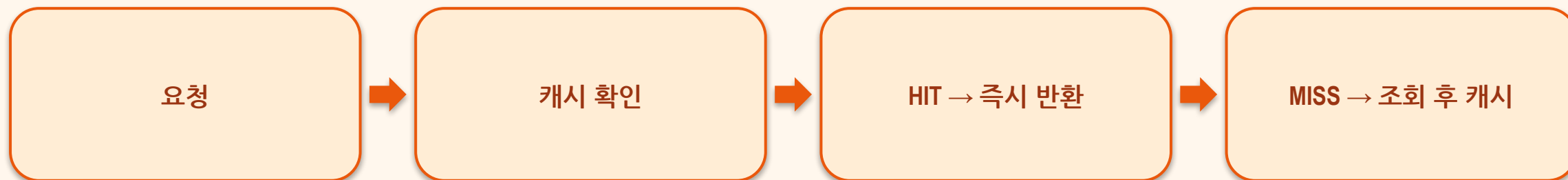
플랫폼이 없어도 죽지 않게 (fallback)

▶ 자체 호스팅 인스턴스 장애에도 서비스 유지 — 운영 신뢰성의 필수 장치



매 호출 왕복 줄이기

▶ `cache_ttl_seconds` — 만료되면 백그라운드에서 조용히 갱신



SECTION 04

앱 연결 + 관찰성 링크

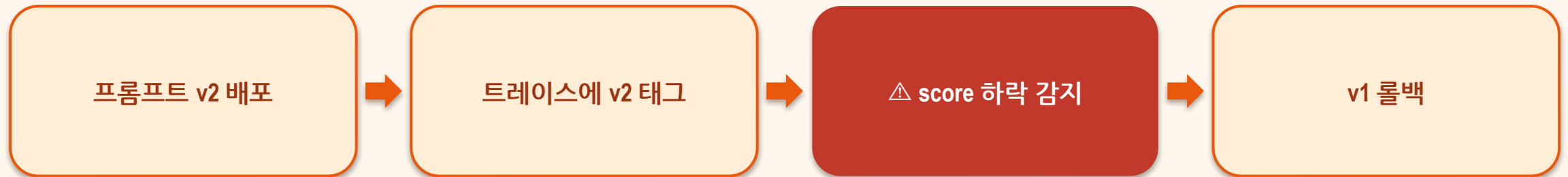
OpenAI 드롭인 + langfuse_prompt

▶ 이 호출의 트레이스에 '어떤 프롬프트 버전'인지 링크가 붙는다

```
from langfuse.openai import openai
prompt = langfuse.get_prompt("recommend-system")
openai.chat.completions.create(
    model=prompt.config["model"],
    messages=[{"role": "system", "content": prompt.compile(area="강남", budget=20000)},
              {"role": "user", "content": "매운 점심"}],
    langfuse_prompt=prompt)          # ← 트레이스에 버전 링크
```

관찰성과 만나는 순간 (18차시 연결)

▶ '어느 프롬프트 버전이 만든 답인지'가 트레이스에 남는다



🔗 관찰성(18) + 프롬프트 관리(19) + 평가(20)가 하나의 루프로

‘정말 더 나은가?’는 평가로 (20차시)

▶ 오늘 만든 버전 관리 체계가 그 실험의 토대

v1 (현재 production)

- 기존 프롬프트
- 기준선 (baseline)

VS

v2 (새 후보)

- 바꾼 프롬프트
- 도전자 (challenger)
- 같은 평가셋으로 Score 비교 → 20차시

SECTION 05

마무리

오늘 3줄

- 프롬프트 = 버전·라벨로 관리하는 자산 (PromptOps) — 하드코딩을 버려라
- create_prompt → get_prompt → compile — 배포=라벨 이동 · 롤백=되돌리기
- 비교는 통계로, 드리프트는 기준선으로 — **langfuse_prompt**로 트레이스에 링크

프롬프트도 코드다 — 버전 없이 바꾸지 마라.

— 하드코딩을 버리면 프롬프트 변경이 안전하고, 빠르고, 비교 가능해진다